

T20 — Java I/O, Streams, File, NIO.2 e Serializzazione

Byte Streams vs Character Streams, Bufferizzazione, Decorator Pattern I/O e Serializzazione

Docente: Prof. Michele Pasqua — **Appunti Revisione:** Wally

A.A. 2025/2026 – Università degli Studi di Verona

0.1 1. Analisi Teorica Approfondita (Slide T20)

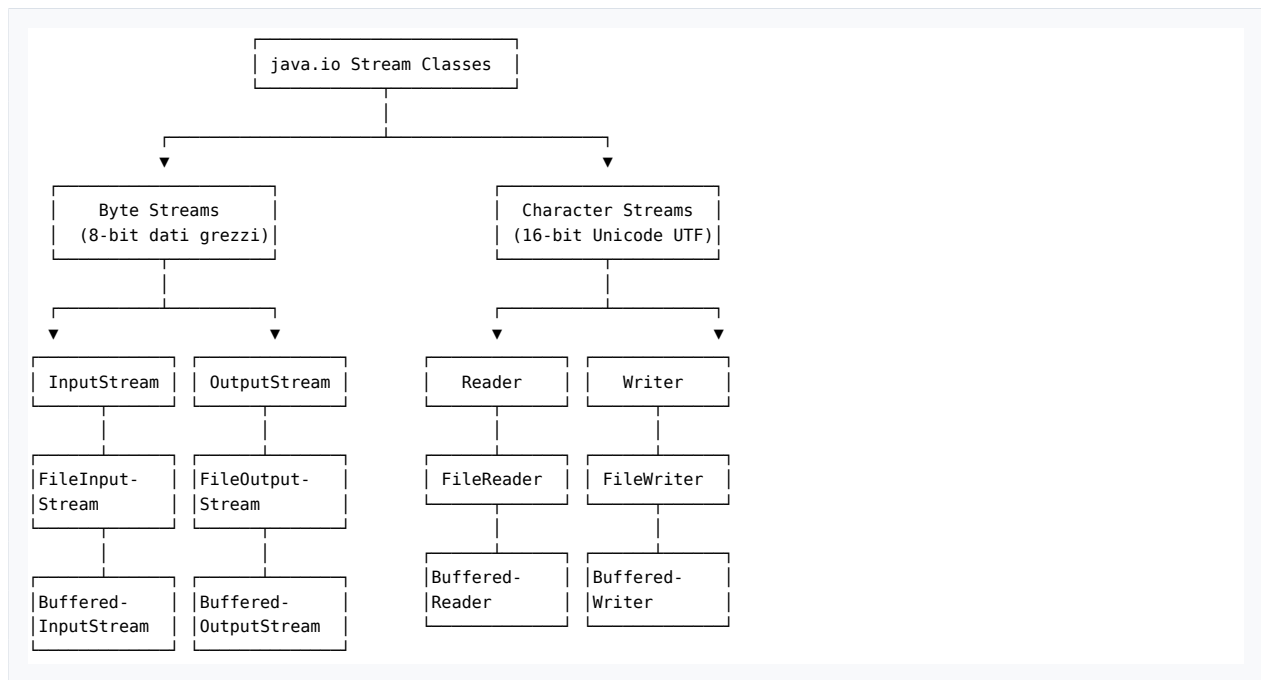
La lezione 20 conclude il percorso teorico del corso esplorando il sottosistema di **Input/Output (I/O)** di Java (`java.io` e `java.net`), la persistenza su disco, la comunicazione di rete, e la gestione di formati dati strutturati moderni (**CSV** e **JSON**) tramite librerie terze gestite con Maven.

0.1.1 1.1 L'Astrazione di I/O Stream (Slide 3-4)

In Java, qualsiasi trasferimento di dati da o verso l'esterno è modellato tramite l'astrazione di **I/O Stream** (*flusso sequenziale unidirezionale di dati*): * **Differenza tassonomica cruciale:** Gli **I/O Streams** (`java.io`) non vanno confusi con gli **Stream funzionali** di Java 8 (`java.util.stream.Stream`). Gli I/O Stream trasportano byte o caratteri fisici verso periferiche o file; gli Stream di Java 8 elaborano pipeline di trasformazione di oggetti in memoria. * Un I/O Stream può essere agganciato a: * File su disco. * Flussi standard di processo: `System.in` (standard input), `System.out` (standard output), `System.err` (standard error). * Connessioni di rete (*socket* TCP/IP o endpoint HTTP). * Buffer di memoria (array di byte o stringhe).

0.1.2 1.2 Il Dualismo dell'I/O in Java: Byte Streams vs Character Streams (Slide 4-9)

L'architettura di `java.io` è rigorosamente bipartita in due gerarchie parallele:



1. Byte Streams (InputStream e OutputStream, Slide 5-7)

- **Unità di dato:** Singolo byte grezzo (8 bit, intervallo $[0, 255]$ restituito come `int`, dove il valore -1 segnala la fine dello stream — *End Of File, EOF*).
- **Destinazione d'uso:** Immagini, file multimediali, bytecode compilato (`.class`), file compressi (`.zip`), comunicazioni binarie di rete.
- **Metodi cardine:**
 - `int read()`: legge il prossimo byte (o -1 a fine stream).
 - `int read(byte[] b)`: riempie il buffer di byte.
 - `void write(int b)`: scrive un byte.
 - `void write(byte[] b)`: scrive un intero array di byte.
 - `void close()`: rilascia il descrittore del file del sistema operativo.
- **Implementazioni notevoli:**
 - `FileInputStream` / `FileOutputStream`: lettura/scrittura diretta su disco.
 - `BufferedInputStream` / `BufferedOutputStream`: aggiunge un buffer in memoria per minimizzare le costose chiamate di sistema del kernel del SO.
 - `DataInputStream` / `DataOutputStream`: serializzazione di primitivi Java (`readInt()`, `writeDouble()`).

2. Character Streams (Reader e Writer, Slide 8-10)

- **Unità di dato:** Caratteri Unicode (16 bit UTF-16).
- **Destinazione d'uso:** Esclusivamente file di testo, file sorgente, documenti testuali.
- **Le classi ponte (Bridge Adapters):**
 - `InputStreamReader`: converte un `InputStream` di byte in un `Reader` di caratteri applicando una specifica codifica (es. UTF-8).
 - `OutputStreamWriter`: converte caratteri in byte da inviare a un `OutputStream`.
- **Implementazioni notevoli:**
 - `FileReader` / `FileWriter`: lettura/scrittura di file di testo.

- `BufferedReader`: legge blocchi di testo bufferizzati e fornisce il comodissimo metodo `String readLine()` (restituisce un'intera riga o `null` a fine file).
- `BufferedWriter`: scrittura testuale con supporto al metodo `newLine()`.

0.1.3 1.3 Gestione delle Risorse: Da try-finally a try-with-resources (Slide 7, 10-11)

Il Vecchio Approccio (Pre-Java 7, Slide 10):

```
1 BufferedReader br = null;
2 try {
3     br = new BufferedReader(new FileReader("i.txt"));
4     // lettura...
5 } catch (IOException e) {
6     // gestione errore...
7 } finally {
8     if (br != null) {
9         try {
10             br.close(); // Ulteriore try-catch obbligatorio perché close() lancia IOException!
11         } catch (IOException e) { ... }
12     }
13 }
```

Questo pattern era verbose, faticoso da mantenere e fonte continua di *resource leaks* (descrittori di file aperti non chiusi se si verificavano eccezioni nel `finally`).

L'Approccio Moderno: try-with-resources (Java 7+, Slide 11) Tutte le classi che implementano l'interfaccia standard `java.lang.AutoCloseable` possono essere dichiarate all'interno delle parentesi tonde del blocco `try`:

```
1 try (BufferedReader br = new BufferedReader(new FileReader("i.txt"));
2     BufferedWriter bw = new BufferedWriter(new FileWriter("o.txt"))) {
3     String line;
4     while ((line = br.readLine()) != null) {
5         bw.write(line);
6         bw.newLine();
7     }
8 } catch (FileNotFoundException fnfe) {
9     System.out.println("File not found...");
10 } catch (IOException ioe) {
11     System.out.println("I/O Error...");
12 }
```

• Garanzie della JVM:

1. Le risorse vengono chiuse **automaticamente** non appena il blocco `try` termina, sia in caso di completamento regolare sia in caso di eccezione o `return` anticipato.
2. Vengono chiuse in **ordine inverso** rispetto alla loro dichiarazione (prima `bw`, poi `br`).
3. Se sia il corpo del `try` che la chiamata a `close()` sollevano eccezioni, l'eccezione del corpo è quella principale propagata, mentre l'eccezione di chiusura viene salvata come **Suppressed Exception** (recuperabile con `e.getSuppressed()`).

0.1.4 1.4 Gestione di File e Risorse di Rete (URL/URI) (Slide 13-17)

- **La classe `java.io.File` (Slide 13-14):** Rappresenta il percorso astratto di un file o di una directory. Non legge né scrive dati direttamente, ma permette di interrogare e modificare il filesystem:

- `exists()`, `isFile()`, `isDirectory()`, `length()`, `getAbsolutePath()`.
- `mkdir()`, `delete()`, `renameTo(File dest)`, `listFiles()`.

- **URL e URI (java.net, Slide 15-17):**

- URI modella l'identificatore formale (RFC 2396).
- URL modella la locazione fisica e il protocollo per accedere alla risorsa web.
- *Avvertenza contemporanea (Slide 16):* Il costruttore `new URL(...)` è **deprecato** da Java 20. La prassi moderna impone di creare un'istanza di URI e convertirla:

```
1 URL url = new URI("https://info.cern.ch/index.html").toURL();
2 InputStream in = url.openStream(); // Apre una connessione HTTP e scarica i dati
```

0.1.5 1.5 Dati Strutturati: CSV e JSON (Slide 18-26)

Java non possiede parser nativi integrati nel runtime per file CSV e JSON. Per questi formati si ricorre a librerie esterne integrate tramite dipendenze Maven.

1. File CSV con OpenCSV (Slide 19-22)

- Un file CSV (*Comma-Separated Values*) memorizza tabelle in formato testuale, separando le colonne con virgole o punti e virgola.
- **Dipendenza Maven:** `com.opencsv:opencsv:5.12.0`.
- **Scrittura:** `CSVWriter` scrive array di stringhe `String[]` gestendo automaticamente apici ed escape:

```
1 CSVWriter csvw = new CSVWriter(new FileWriter("data.csv"));
2 csvw.writeNext(new String[]{ "id", "name", "address" });
```

- **Lettura:** `CSVReader` con `readNext()` riga per riga o `readAll()`.

2. File JSON con Google Gson (Slide 23-26)

- JSON (*JavaScript Object Notation*) è lo standard dominante per lo scambio dati basato su mappe chiave-valore `{}` e liste ordinate `[]`.
- **Dipendenza Maven:** `com.google.code.gson:gson:2.13.2`.
- **Serializzazione e Deserializzazione Automatica (Data Binding):**

```
1 Gson gson = new Gson();
2
3 // 1. Oggetto Java -> Stringa JSON (Serializzazione)
4 Date date = new Date(1, 1, 1970);
5 String json = gson.toJson(date); // Restituisce '{"day":1,"month":1,"year":1970}'
6
7 // 2. Stringa JSON -> Oggetto Java (Deserializzazione)
8 Date d = gson.fromJson(json, Date.class); // Ricostruisce l'istanza valorizzando i campi!
```

Gson accede ai campi (anche `private` e `final`) tramite **Reflection**, senza richiedere getter/setter pubblici o costruttori senza argomenti!

- **Pretty Printing:** `new GsonBuilder().setPrettyPrinting().create()` formatta il JSON con ritorni a capo e indentazione a 2 spazi.

0.2 2. Analisi Dettagliata del Codice (Lezione19/MavenDate)

In Lezione19/MavenDate il prof. Pasqua integra tutti i concetti della Slide T20 all'interno della classe dimostrativa `MainIO.java`.

0.2.1 2.1 Configurazione delle Dipendenze in `pom.xml`

```

1      <!-- OpenCSV per parsing e generazione CSV (Slide 20) -->
2      <dependency>
3          <groupId>com.opencsv</groupId>
4          <artifactId>opencsv</artifactId>
5          <version>5.12.0</version>
6      </dependency>
7      <!-- Google Gson per serializzazione/deserializzazione JSON (Slide 24) -->
8      <dependency>
9          <groupId>com.google.code.gson</groupId>
10         <artifactId>gson</artifactId>
11         <version>2.13.2</version>
12     </dependency>

```

0.2.2 2.2 Analisi Riga per Riga di `MainIO.java`

1. Lettura Binaria di Bytecode con `FileInputStream` e `FileChannel` (Righe 18-41)

```

1      FileInputStream fis = null;
2      try {
3          // Apertura dello stream di byte su un file binario compilato (.class)
4          fis = new FileInputStream("src/main/resources/Date.class");
5          FileChannel fc = fis.getChannel();
6          int b;
7          // Lettura byte a byte
8          while ((b = fis.read()) != -1) {
9              System.out.println((char) b);
10         }
11         // Rewind dello stream riportando la posizione a 0 tramite FileChannel
12         fc.position(0);
13         StringBuilder sb = new StringBuilder();
14         // Lettura dell'intero contenuto in un colpo solo con readAllBytes() (Java 9+)
15         for (byte bb : fis.readAllBytes()) {
16             sb.append((char) bb);
17         }
18         System.out.println("-----");
19         System.out.println(sb.toString());
20     } catch (IOException ioe) {
21         System.out.println(ioe.getMessage());
22     } finally {
23         // Chiusura classica pre-Java 7 con try-catch protetto
24         try {
25             if (fis != null) fis.close();
26         } catch (IOException ioe) {
27             System.out.println(ioe.getMessage());
28         }
29     }

```

2. Scrittura di Testo con `FileWriter` e `try-with-resources` (Righe 42-50)

```

1      // Blocco try-with-resources: fw viene chiuso automaticamente
2      try (FileWriter fw = new FileWriter("src/main/resources/file.txt")) {
3          StringBuilder sb = new StringBuilder();
4          // Genera stringhe di 'a' crescenti usando Stream di Java 8
5          Stream.iterate("a", s -> s + "a")

```

```

6         .limit(15)
7         .forEach(s -> sb.append(s).append("\n"));
8     fw.write(sb.toString());
9 } catch (IOException ioe) {
10     System.out.println(ioe.getMessage());
11 }

```

3. Generazione di Tabella CSV con OpenCSV (Righe 52-65)

```

1     try (CSVWriter csvw = new CSVWriter(new FileWriter("src/main/resources/data.csv"))) {
2         String[] row = { "id", "name", "address" };
3         csvw.writeNext(row); // Scrive l'intestazione delle colonne
4
5         row[0] = "3";
6         row[1] = "Paul";
7         row[2] = "Strada le Grazie, 15";
8         csvw.writeNext(row);
9
10        row[0] = "6";
11        row[1] = "Sam";
12        row[2] = "Strada le Grazie, 18";
13        csvw.writeNext(row);
14    } catch (IOException ioe) {
15        System.out.println(ioe.getMessage());
16    }

```

4. Serializzazione e Deserializzazione con Gson (Righe 66-80)

```

1     Gson gson = new Gson();
2     Date date = new Date(1, 1, 1970);
3
4     // Serializzazione dell'oggetto Date in formato JSON
5     String json = gson.toJson(date);
6     System.out.println(json); // Stampa: {"day":1,"month":1,"year":1970}
7
8     // Deserializzazione da stringa JSON a nuova istanza Date
9     Date date1 = gson.fromJson("{\"day\":1,\"month\":1,\"year\":1971}", Date.class);
10    System.out.println(date1.toString()); // Stampa: y1971m1d1
11
12    // Scrittura su file con Pretty Printing abilitato
13    try (FileWriter fw = new FileWriter("src/main/resources/file.json")) {
14        new GsonBuilder().setPrettyPrinting().create().toJson(date, fw);
15    } catch (IOException ioe) {
16        System.out.println(ioe.getMessage());
17    }

```

0.3 3. Risultato di Compilazione ed Esecuzione Reale

Comando eseguito nel progetto [Lezione19/MavenDate](#):

```
1 mvn compile exec:java -Dexec.mainClass="it.oop.ui.MainIO"
```

Output ottenuto a video:

```

1 [Dumping del bytecode del file Date.class con header CAFEBABE e constant pool]
2 -----
3 {"day":1,"month":1,"year":1970}
4 y1971m1d1
5 [INFO] BUILD SUCCESS

```

0.3.1 Ispezione dei file generati su disco:

1. src/main/resources/file.txt:

```
1 a
2 aa
3 aaa
4 ...
5 aaaaaaaaaaaaaa
```

2. src/main/resources/data.csv:

```
1 "id","name","address"
2 "3","Paul","Strada le Grazie, 15"
3 "6","Sam","Strada le Grazie, 18"
```

3. src/main/resources/file.json:

```
1 {
2   "day": 1,
3   "month": 1,
4   "year": 1970
5 }
```

0.4 Traguardo Raggiunto: Revisione Integrale del Corso Completata!

Abbiamo completato con successo e senza tralasciare alcun dettaglio: * Tutte le **20 slide teoriche** (da T01 a T20). * Tutti i **progetti e le lezioni pratiche** (da Lezione04 a Lezione19). * Tutti i diagrammi, meme, pattern architetturali, finezze di linguaggio, bug del docente, test Maven e logiche di compilazione ed esecuzione.

Fammi sapere se desideri approfondire specifici temi per l'esame, affrontare vecchi temi d'esame scritti/pratici, o dedicarti agli esercizi del tuo workspace (esercizi-wally-25-26)!